COLEGIUL NAȚIONAL DE INFORMATICĂ

PIATRA-NEAMȚ

Atestat pentru obținerea competențelor profesionale a absolvenților claselor de matematică-informatică și matematică-informatică, intensiv informatică

Tema proiectului: Tic-Tac-Toe

Prof. îndrumător

Alterescu Cristina

Elev,

Țuburlui Mihai

Mai 2026

Tic-Tac-Toe

Descriere

Acesta este un joc clasic de Tic-Tac-Toe (X și 0) implementat în C++ cu o interfață grafică folosind OpenGL și GLUT. Jocul include un AI (Inteligență Artificială) care poate juca la diferite niveluri de dificultate, făcând jocul potrivit atât pentru începători, cât și pentru jucătorii experimentați.

Cum funcționează jocul

Tic-Tac-Toe este un joc pentru doi jucători care se joacă pe o tablă de 3x3. Jucătorii alternează plasând simbolurile (X și 0) pe tablă. Scopul este să obții trei simboluri identice într-un rând, coloană sau diagonală.

Reguli de bază:

- Tabla are 9 pătrate (3x3).
- Primul jucător folosește X, al doilea 0.
- Jucătorii alternează turele.
- Jocul se termină când:
  - Un jucător obține trei simboluri în linie (câștig).
  - Toate pătratele sunt ocupate fără câștigător (egalitate).

Interfața grafica

Caracteristici

Moduri de joc:

- **Jucător vs Jucător**: Doi jucători umani joacă unul împotriva celuilalt.
- **Jucător vs AI**: Un jucător uman joacă împotriva calculatorului.

Opțiuni AI:

- **Activare AI**: Poți activa sau dezactiva AI-ul.
- **Simbol AI**: Alege dacă AI-ul joacă cu X sau cu 0.
- **Dificultate**:
  - **Easy**: AI-ul are o șansă de 40% să facă o mutare perfectă.
  - **Hard**: AI-ul are o șansă de 90% să facă o mutare perfectă.
  - **Impossible**: AI-ul execută constant o mutare perfectă.

Variabile relevante:

Matricele de stare a jocului:

- **int matrix[3][3]**: Matricea principală a jocului care stochează starea fiecărei celule din tablă. Valorile posibile sunt:
  - **0** - celula este goală/neocupată
  - **1** - celula conține semnul X
  - **2** - celula conține semnul 0 (zero)
- Utilizare în cod:

-
```
// Exemplu: plasarea unui semn pe tablă
if (matrix[row][col] == 0) {  // Dacă celula este goală
    matrix[row][col] = turn + 1;  // Plasează X (1) sau 0 (2)
}
```

- **int gold[3][3]**: Matricea marcajelor pentru câștig. Aceasta este folosită pentru a evidenția vizual (cu culoare aurie) liniile, coloanele sau diagonalele care formează o combinație câștigătoare. Valorile sunt 0 (neactiv) sau 1 (activ - se afișează în culoare aurie).

Variabile de control al turului și stării:

- **bool turn**: Variabilă booleană care memorează al cui jucător este rândul curent. Valori:
  - **false (0)** - rândul jucătorului X
  - **true (1)** - rândul jucătorului 0
- Exemplu:
- `turn = !turn;  // Alternează tururile între jucători`
- **bool firstTurn**: Flag care verifică dacă AI-ul și-a făcut o mutare. Utilizat pentru a optimiza prima mișcare a AI-ului (IA joacă centrul tablei).
- **int state**: Starea curentă a jocului, care poate fi:
  - **0** - jocul continuă (nici un câștigător)
  - **1** - X a câștigat
  - **2** - 0 (zero) a câștigat
  - **3** - egalitate (tabla plină, niciun câștigător)

Variabile pentru configurația AI-ului:

- **bool AI**: Variabilă care determină modul de joc:
  - **true** - Jucător vs AI (computer-ul joacă automat)
  - **false** - Jucător vs Jucător (doi jucători umani)
- **bool AIXor0**: Determină semnul cu care joacă AI-ul:
  - **false** - AI-ul joacă cu semnul X (1)
  - **true** - AI-ul joacă cu semnul 0 (2)
- **int AIDifficulty**: Nivelul de dificultate al AI-ului:
  - **0** - Easy: AI-ul are 40% șansă să facă o mișcare perfectă, 60% șansă o mișcare aleatorie
  - **1** - Hard: AI-ul are 90% șansă să facă o mișcare perfectă, 10% șansă o mișcare aleatorie
  - **2** - Impossible: AI-ul execută mereu mișcarea optimă folosind algoritmul Minimax

Variabile de interfață:

- **bool settings**: Flag care controlează afișajul meniului de setări:
  - **true** - meniul de setări este deschis (se afișează butoanele de configurare)
  - **false** - meniul de setări este închis (se joacă jocul pe tablă)

Funcții relevante:

Funcții de utilitate și detecție:

- **bool chance(int percent)**: Generează un număr aleator între 1 și 100 și compară cu valoarea percent pentru a crea comportament probabilistic. Returnează true dacă numărul generat este ≤ percent, altfel false.
- Implementare:
- ```
  int pInterface::chance(int percent) {
    std::random_device rd;
    std::mt19937 mt(rd());  // Generator Mersenne Twister
    std::uniform_int_distribution<int> dist(1, 100);
    return dist(mt) <= percent;
  }
  ```
- **bool eq3(int a, int b, int c)**: Funcție helper care verifică dacă trei valori sunt egale și toate sunt diferite de 0. Este utilizată pentru detectarea câștigurilor (trei X sau trei 0 pe rând).
- ```
  bool pInterface::eq3(int a, int b, int c) {
    return a == b && b == c && a != 0;
  }
  ```
- **std::pair<int, int> getCursorPos()**: Convertește coordonatele pixelului cursorului mouse în coordonata pe tabla de 3x3. Divizează poziția mouse-ului cu dimensiunea unei celule pentru a determina linia și coloana.

Funcții de management al stării jocului:

- **void drawSettings()**: Comută afișajul meniului de setări și rerendeează ecranul.
- **void drawGrid()**: Randeaza tabla de joc cu simbolurile X și 0 plasate pe tablă. Desenează gridurile și marchează celulele câștigătoare cu culoare aurie dacă jocul s-a terminat.
- **void resetGame()**: Aduce jocul la starea inițială:
  - Golește matricea principală și matricea gold
  - Resetează starea (state = 0)
  - Seteaza turele inițiale bazate pe configurația AI
-
```
void pInterface::resetGame() {
  for (int i = 0; i < 3; ++i)
    for (int j = 0; j < 3; ++j)
      matrix[i][j] = 0, gold[i][j] = 0;
  state = 0;
  turn = AI ? AIXor0 : false;
  firstTurn = true;
}
```
- **void handleClick()**: Gestionează clicurile mouse-ului și logica alternării turelor. Dacă este rândul AI-ului, cheamă funcția corespunzătoare de dificultate.
- **int detectLocalState(int mat[3][3])**: Verifică starea unei matrici date și returnează:
  - **0** - jocul continuă

- **1** - X a câștigat
- **2** - 0 a câștigat
- **3** - egalitate

- Implementare:

```
int pInterface::detectLocalState(int mat[3][3]) {
  // Verifică rânduri și coloane
  for (int i = 0; i < 3 && !localState; ++i) {
    if (eq3(mat[i][0], mat[i][1], mat[i][2]))
      localState = mat[i][0];
    if(eq3(mat[0][i], mat[1][i], mat[2][i]))
      localState = mat[0][i];
  }
  // Verifică diagonalele
  if (!localState && (eq3(mat[0][0], mat[1][1], mat[2][2])))
    localState = mat[1][1];
  if (!localState && (eq3(mat[0][2], mat[1][1], mat[2][0])))
    localState = mat[1][1];
  // Verifică egalitate
  if (!localState) { ... }
  return localState;
}
```

- **void detectState(bool showOutput = false)**: Similar cu detectLocalState, dar operează pe matricea principală. Dacă showOutput este true, marchează celulele câștigătoare în matricea gold.

Funcții ale algoritmului de inteligență artificială:

- **int minimax(int mat[3][3], int d, bool player)**: Algoritm de inteligență artificială care evaluează toate mutările posibile și calculează cel mai bun scor. Acesta este un algoritm de backtracking care examineaza arborele jocului.

- Caracteristici:

  - **Recursive**: Examineaza fiecare stare posibilă a jocului
  - **d (depth)**: Parametrul de adâncime, ajutat pentru a penaliza mutări târzii
  - **player**: Determină al cui tur este (true = AI, false = oponent)

-
```
int pInterface::minimax(int mat[3][3], int d, bool player) {
    int localState = detectLocalState(mat);
    // Cazuri de bază: joc terminat
    if (localState != 0) {
        if (localState == 1) return AIXor0 ? d - 10 : 10 - d;
        if (localState == 2) return AIXor0 ? 10 - d : d - 10;
        return 0;  // egalitate
    }

    int bestScore = player ? INT_MIN : INT_MAX;
    // Evaluează toate mutările posibile
    for (int i = 0; i < 3; ++i)
        for (int j = 0; j < 3; ++j)
        if (mat[i][j] == 0) {
            mat[i][j] = player ? (AIXor0 + 1) : (AIXor0 ? 1 : 2);
            int score = minimax(mat, d + 1, !player);
```

```
            // Maximization pentru AI, minimization pentru oponent
            if (player) bestScore = std::max(score, bestScore);
            else bestScore = std::min(score, bestScore);
            mat[i][j] = 0;  // Undo mutare
        }
    return bestScore;
}
```

- **void generateRandomMove()**: Generează o mutare aleatorie pe o celulă goală din tablă.
- **void AIEasy()**: Implementează nivelul Easy cu distribuția: 40% mișcări perfecte, 60% mișcări aleatoare.
-
```
void pInterface::AIEasy() {
    if (chance(40)) AIImpossible();  // 40% probabilitate
    else generateRandomMove();       // 60% probabilitate
}
```
- **void AIHard()**: Implementează nivelul Hard cu distribuția: 90% mișcări perfecte, 10% mișcări aleatoare.
-
```
void pInterface::AIHard() {
    if (chance(90)) AIImpossible();   // 90% probabilitate
    else generateRandomMove();        // 10% probabilitate
}
```
- **void AIImpossible()**: Implementează nivelul Impossible, făcând întotdeauna mișcarea optimă prin evaluarea tuturor posibilităților cu algoritmul Minimax. În prima mutare, AI-ul alege centrul tablei (1,1) pentru performanță optimă.

```
void pInterface::AIImpossible() {
    // Prima mutare: joacă centrul
    if (firstTurn) {
        firstTurn = !firstTurn;
        if (matrix[1][1] == 0) {
            matrix[1][1] = AIXor0 + 1;
        }
        return;
    }

    int bestScore = INT_MIN;
    std::pair<int, int> bestMove;

    // Evaluează fiecare mutare posibilă
    for (int i = 0; i < 3; ++i)
        for (int j = 0; j < 3; ++j)
            if (matrix[i][j] == 0) {
                matrix[i][j] = AIXor0 + 1;
                int score = minimax(matrix, 0, false);
                if (bestScore < score) {
                    bestScore = score;
                    bestMove = { i, j };
                }
                matrix[i][j] = 0;  // Undo
            }
    matrix[bestMove.first][bestMove.second] = AIXor0 + 1;  // Execută
}
```

Interfață grafică:

- Interfață intuitivă cu butoane și elemente vizuale.
- Suport pentru redimensionarea ferestrei.
- Culori atractive pentru elementele de joc.

Informații pentru admiterea la informatica

Acest proiect demonstrează competențe avansate în programare și informatică, fiind potrivit pentru portofoliul de admitere la facultățile de informatică sau pentru concursuri de programare. Principalele aspecte educaționale și tehnice acoperite includ:

Competențe demonstrare:

- **Programare în C++**: Utilizarea limbajului C++ pentru dezvoltarea unei aplicații complete, inclusiv gestionarea memoriei, pointeri și clase.
- **Algoritmi și structuri de date**: Implementarea algoritmului Minimax pentru inteligența artificială, demonstrând înțelegerea algoritmilor de căutare și optimizare.
- **Inteligență artificială**: Dezvoltarea unui AI care joacă la diferite niveluri de dificultate, folosind probabilități și decizii strategice.
- **Grafică computerizată**: Utilizarea OpenGL și GLUT pentru crearea unei interfețe grafice interactive, inclusiv randarea 2D și gestionarea evenimentelor mouse și tastatură.
- **Arhitectura software**: Organizarea codului în module separate (graphics, interface), demonstrând principiile bune de proiectare software.
- **Rezolvarea problemelor**: Implementarea logicii jocului Tic-Tac-Toe, inclusiv detectarea stărilor de câștig și egalitate.

Valoarea proiectului:

- Demonstrează abilitatea de a dezvolta aplicații complexe de la zero.
- Include documentație detaliată și explicații ale algoritmilor utilizați.
- Poate fi extins cu funcționalități suplimentare pentru a demonstra și mai multe competențe.
- Este un exemplu practic de aplicare a conceptelor teoretice învățate în informatică.

Exemple de cod și concepte avansate pentru atestatul la informatică:

1. Utilizarea tablourilor 2D și indexării:

```
// Declarație și inițializare
int matrix[3][3]{};  // Inițializare implicită cu 0

// Acces la elemente
for (int i = 0; i < 3; ++i)
    for (int j = 0; j < 3; ++j)
        matrix[i][j] = 0;  // Iterare 2D
```

2. Tipul de dată std::pair - structură de dată:

```
#include <utility>  // Pentru std::pair
std::pair<int, int> pos = { 1, 2 };  // Pereche de două întregi
int row = pos.first;   // Acces la primul element
int col = pos.second;  // Acces la al doilea element
```

3. Utilizarea bibliotecii <random> pentru generarea numerelor aleatoare:

```
#include <random>

int chance(int percent) {
```

```
    std::random_device rd;            // Seed-ul aleator
    std::mt19937 mt(rd());            // Generator Mersenne Twister
    std::uniform_int_distribution<int> dist(1, 100);  // Distribuție uniformă
    return dist(mt) <= percent;       // Comparare cu percent
}
```

4. Valori limitate din <climits>:

```
#include <climits>

int bestScore = INT_MIN;  // Cea mai mică valoare negativă
int worstScore = INT_MAX; // Cea mai mare valoare pozitivă
bestScore = std::max(score, bestScore);  // Maximization în Minimax
```

5. Structuri și clase în C++:

```
class pInterface {
    public:
    int matrix[3][3]{};   // Variabile membre
    bool turn = false;
    bool AI = true;

    // Metode
    void handleClick();
    int minimax(int mat[3][3], int d, bool player);

    private:
    // Constructor și destructor
    pInterface();
    ~pInterface();
};
```

6. Operatori logici și boolean:

```
// Operatori logici în condiții
if (matrix[pos.first][pos.second] == 0 && state == 0) {
    // Dacă celula este goală ȘI starea este "continuă"
}

if (AI && !AIXor0) {  // AI ON ȘI AI joacă cu X
    handleClick();
}

turn = !turn;  // Negare boolean
AIXor0 = !AIXor0;  // Schimbare
```

7. Operatori ternari și transformări:

```
// Operator terniar - condiție ? valoare_true : valoare_false
state = (localState == 1) ? 1 : (localState == 2) ? 2 : 0;

// Expresii ternare nested
std::string text = turn ? "Turneul jucătorului 0" : "Turneul jucătorului X";
```

8. Recursivitate în Minimax:

```
// Apel recursiv - examineaza fiecare stare posibilă
for (int i = 0; i < 3; ++i) {
    for (int j = 0; j < 3; ++j) {
        if (mat[i][j] == 0) {
            mat[i][j] = player ? (AIXor0 + 1) : (AIXor0 ? 1 : 2);
            int score = minimax(mat, d + 1, !player);  // Apel recursiv
            mat[i][j] = 0;  // Backtracking - anularea mutării
        }
```

```
    }
}
```

9. Algoritmi - Minimax (game tree search):

Minimax este un algoritm adversarial care: - **Explorează** toate mutările posibile până la sfârșitul jocului - **Evaluează** fiecare stare terminală (câștig, pierdere, egalitate) - **Propagă** scorurile înapoi prin arbore - **Selectează** cea mai bună mutare pe baza scorului maxim al jucătorului AI - **Complexitate**: **O(b^d)** unde b = numărul de mutări posibile, d = adâncimea

```
int minimax(int mat[3][3], int d, bool player) {
    int localState = detectLocalState(mat);

    // Cazuri de bază: joc terminat
    if (localState != 0) {
        if (localState == 1) return AIXor0 ? d - 10 : 10 - d;
        if (localState == 2) return AIXor0 ? 10 - d : d - 10;
        return 0;  // egalitate
    }

    int bestScore = player ? INT_MIN : INT_MAX;

    // Evaluează toate mutările posibile
    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 3; ++j) {
            if (mat[i][j] == 0) {
                mat[i][j] = player ? (AIXor0 + 1) : (AIXor0 ? 1 : 2);
                int score = minimax(mat, d + 1, !player);  // Recursiune
```

```
            // Maximization pentru AI, minimization pentru oponent
            if (player)
                bestScore = std::max(score, bestScore);
            else
                bestScore = std::min(score, bestScore);

            mat[i][j] = 0;  // Backtracking
        }
    }
  }
  return bestScore;
}
```

10. Pointers și transmitere de parametri:

```
// Transmitere prin pointer pentru modificare directă
void minimax(int mat[3][3], int d, bool player) {
  mat[0][0] = 1;  // Modifică apoi tabloul original
}

// Transmitere prin valoare (copie)
void display(int matrix[3][3]) {
  // Orice schimbare afectează original (deoarece array-urile se transmit prin referință)
}
```

11. Optimizări și bune practici:

```
// Memoization - evitarea recalculărilor (pentru versiuni mai avansate)
std::map<GameState, int> memo;  // Cache pentru scoruri Minimax

// Early termination - ieșire din loop dacă condiție este îndeplinită
```

```
for (int i = 0; i < 3 && !state; ++i) {  // !state = continuă doar dacă jocul nu s-a terminat
    if (eq3(matrix[i][0], matrix[i][1], matrix[i][2])) {
        state = matrix[i][0];  // Găsit câștigător, ieșire
    }
}

// Probabilități și controlare AI
if (chance(90))     // 90% probabilitate
    AIImpossible();  // Execută mutare perfectă
else
    generateRandomMove();  // 10% probabilitate de mutare aleatorie
```

Cum să compilezi și să rulezi:

Cerințe de sistem:

- Sistem de operare: Linux sau Windows.
- Compilator: g++ (pentru a compila pentru Linux) sau MinGW-w64 (pentru a compila pentru Windows) sau Visual Studio (pentru Windows).
- Bibliotecile sunt incluse în directorul /lib/ și header-ele în /include/: OpenGL, GLUT, GLU, GLEW, DevIL (IL).

Compilare pe Linux:

- Proiectul conține două scripturi de compilare:
    1. linux_compile.sh
    2. windows_compile.sh

1. Deschide terminalul in directurul principal al proiectului.
2. Execută scriptul de compilare specific sistemului ales.

Compilare pe Windows:

1. Deschide Visual Studio.
2. Deschide proiectul Tic-Tac-Toe.sln.
3. Construiește proiectul (Build > Build Solution).

Rulare:

- Pe Linux: ./Tic-Tac-Toe
- Pe Windows: Rulează executabilul generat.

Controale:

- **Click stânga**: Plasează simbolul pe tablă sau interacționează cu butoanele din setări.
- **ESC**: Deschide/închide meniul de setări sau resetează jocul.

Structura proiectului

- main.cpp: Punctul de intrare al programului, gestionează evenimentele și randarea.
- main.hpp: Header-ul principal.
- structs.hpp: Structuri de date folosite în proiect.
- interface/: Codul pentru logica jocului și interfață.
- graphics/: Componente grafice (buton, checkbox, imagine, etc.).
- include/: Header-e pentru OpenGL, GLUT, DevIL.
- lib/: Biblioteci statice.
- linux_compile.sh, windows_compile.sh: Scripturi de compilare.

Autor

Acest proiect a fost realizat de **Țuburlui Mihai** de la **Colegiul Național de Informatică Piatra-Neamț**.